

Lenguajes imperativos y declarativos. El paradigma orientado a objetos. Comparación con lenguajes imperativo, lógico y funcional

Índice

Lenguajes de programación imperativos. Programación imperativa	2
Lenguajes de programación declarativos. Programación declarativa	2
Cómo sería pedir una pizza según uno y otro paradigma	3
Derivaciones de la programación imperativa.....	3
Programación estructurada.....	3
Programación procedural	4
Programación orientada a objetos	4
Otras	4
Derivaciones de la programación declarativa	4
Programación lógica	4
Programación funcional	5
Domain Driven Design	5

Lenguajes de programación imperativos. Programación imperativa

Los lenguajes imperativos definen al paradigma imperativo de programación. En este paradigma se usa el lenguaje para describir cómo cambia el estado del sistema. Típicamente, hay instrucciones (imposiciones) ordenadas en forma de líneas de texto, una debajo de la otra, que se ejecutan de forma secuencial. El lenguaje imperativo describe cómo opera el sistema. La programación imperativa es la más “natural”, dado que surge desde el mismísimo hardware, donde se ejecuta instrucciones de máquina. Estas instrucciones, en su más bajo nivel, mueven registros de datos de un lugar a otro, alteran valores, etc. Es decir, cambian el estado de los integrados de la memoria, del procesador, en fin, de nuestra computadora. A medida que subimos de nivel, los lenguajes siguen haciendo lo mismo, solo que con estructuras más complejas como variables, instrucciones complejas (como bucles, modificadores de visibilidad, e incluso cosas más complejas como las anotaciones).

Lenguajes de programación declarativos. Programación declarativa

Los lenguajes declarativos, en el paradigma declarativo de programación, se utilizan para describir la estructura del sistema y describir qué hace el sistema. Una vez descripta “la misión” del programa, el cómo se llega al resultado depende de la implementación interna del propio lenguaje. Generalmente, se define a la programación declarativa como “no-imperativa”. Es decir, es todo lo que la imperativa no es. En este sentido, la programación declarativa dice qué debe hacer el sistema, en donde la ejecución no afecte otras partes del sistema y pueda representarse como una serie de premisas matemáticas (funciones/transformaciones).

En un lenguaje imperativo, entonces, el sistema pasará de un estado a otro ejecutando una serie de pasos, un algoritmo. En un lenguaje declarativo se debe describir el estado inicial, el estado final y luego el propio motor del lenguaje hará todo lo necesario para ir de un estado al otro.

Ejemplos de lenguajes imperativos hay muchos: C, C++, Java, Perl y un infinito etcétera. En todos estos lenguajes se escriben líneas de código que describen los pasos de un algoritmo para lograr un resultado.

El ejemplo más claro de un lenguaje declarativo es SQL. En SQL se describe qué información debe extraerse de una o más tablas, sin control sobre la forma de obtenerlo. Cómo se obtiene lo maneja el motor de la base. Otro ejemplo ubicuo es HTML. En HTML se describe, por ejemplo, que debe existir cierto texto, centrado y en negrita, sin especificar ningún tipo de lógica ni control de flujo. Cómo es que veremos el texto centrado y en negrita dependerá de cada navegador.

Cómo sería pedir una pizza según uno y otro paradigma

Imperativo	Declarativo
1. Entrar a la pizzería. 2. Esperar a quien atiende. 3. Pedir pizza. 4. Seleccionar gusto. 5. Pagar. 6. Tomar la pizza y salir del local.	1. Quiero una pizza grande de mozzarella para llevar.
Digo paso a paso qué hacer.	Digo qué quiero y dejo que el lenguaje o plataforma llegue al resultado.

Derivaciones de la programación imperativa

El paradigma imperativo da lugar a, al menos, tres tipos de programación imperativa (a veces llamados paradigmas, dada la extensión de su uso o su popularidad).

Programación estructurada

En ella las imposiciones se ven divididas en estructuras. Pueden ser estructura de control de flujo (*if/else*), de repeticiones (*for/while*) o pueden ser estructuras de ordenamiento (bloques de código, generalmente delimitados por TABs o por llaves o corchetes).

Programación procedural

En ella las imposiciones o instrucciones se pueden agrupar en “procedimientos”. Cada procedimiento es un bloque (la programación procedural es una forma de programación estructurada) que se dedica a un cambio de una parte específica del estado del sistema. Estos bloques son conocidos típicamente como “funciones” o “subrutinas”.

Programación orientada a objetos

En ella las instrucciones tratan de modelar un sistema en base a la realidad, haciendo más cercanos los conceptos que vemos en ella. Es por eso que en la programación orientada a objetos aparecen conceptos de clase, objeto, atributo, método, etc., que en el fondo no dejan de ser bloques de código. Asimismo, cabe notar que también hacen uso de estructuras de control y repetición.

Otras

Hay otras derivaciones, como la programación reactiva, que está ganando terreno, pero dependiendo el lenguaje que se use cae dentro de la categoría procedural u orientada a objetos. En todos los casos, un lenguaje imperativo puede hacer uso de todas las formas (JAVA o C++, por ejemplo son lenguajes imperativos orientados a objetos, pero usan bloques, control de flujo y agrupan operaciones en procedimientos llamados “métodos”).

Derivaciones de la programación declarativa

La programación declarativa, al igual que la imperativa, es más que nada un término general. Por eso, tiene varias derivaciones, llamados paradigmas o sub-paradigmas. Al igual que en la programación imperativa, el término “paradigma” surge de la popularidad o extensión en el uso. Las dos más importantes, aunque no las únicas, son:

Programación lógica

En la programación lógica se definen reglas. La programación lógica se basa en el uso de la lógica formal, en la que se ponen premisas, condiciones y resultados. La programación

lógica se expresa mediante formulaciones o transformaciones, declaración de hechos, e interrogantes. Expresiones de tipo:

Hecho: “El profesor es un humano”.

Regla: “Todos los humanos son mortales”

Interrogante: “¿Es el profesor mortal?”

Formalmente, si ‘p’ y ‘q’, entonces ¿‘u’?:

$$(p \wedge q) \rightarrow u$$

Ejemplo en Prolog (uno de los lenguajes lógicos más populares):

humano(profesor).

mortal(X) :- humano(X).

?- mortal(profesor).

Programación funcional

Es también considerado un paradigma, en el cual se definen funciones matemáticas que trabajan como expresiones o declaraciones; no son imposiciones, no son pasos que van alterando el estado. En el código funcional, el resultado depende siempre de los valores de entrada o argumentos, cumpliendo con la premisa de que en la programación declarativa la ejecución de una parte del código no afecte a otras. Es decir, siempre que se llame a una misma función, con los mismos argumentos, arrojará el mismo resultado.

Domain Driven Design

También es muy importante el Domain Driven Design, que es una programación fuertemente orientada a dominios particulares. En el diseño orientado a dominios (según su nombre correcto en castellano) se usan los lenguajes específicos de dominios o DSLs. Los DSLs son lenguajes creados específicamente para cubrir una necesidad específica,

como son las expresiones regulares. Otro ejemplo típico sería SQL. Aquí ocurre algo similar que con la programación orientada a objetos, donde si bien suele considerarse un paradigma por sí mismo, toma elementos de otros.